

# Cuestionario general de la materia

Testing de Software — UNRC

Trabajos prácticos TP1 a TP9

---

Tomás Rodeghiero — Material de repaso final — 2026

Este cuestionario reúne **90 preguntas teóricas con sus respuestas** (10 por unidad), organizadas en los nueve bloques temáticos correspondientes a los trabajos prácticos. Cada respuesta incluye la *explicación conceptual*, una *referencia a la teoría de la cátedra* y un *ejemplo práctico breve*. Está pensado como material imprimible para el repaso final de la materia.

## Índice

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 1. Vocabulario del testing, RIPR, AAA y contratos de equals/hashCode | 1  |
| 2. Automatización con JUnit, data-driven testing y repOK             | 5  |
| 3. Input Space Partitioning (ISP)                                    | 8  |
| 4. Testing basado en grafos                                          | 12 |
| 5. Cobertura lógica de predicados                                    | 15 |
| 6. Testing de mutación y fuzzing                                     | 18 |
| 7. Property-Based Testing (PBT)                                      | 22 |
| 8. Generación automática de tests y <i>mocking</i>                   | 25 |
| 9. Testing de sistemas con estado, no-determinismo e integración     | 29 |
| 10. Resumen de cobertura                                             | 32 |

## 1. Vocabulario del testing, RIPR, AAA y contratos de equals/hashCode

### Temas abordados

- Verificación y Validación (V&V), niveles de testing.
- Vocabulario: *fault*, *error*, *failure*.
- Modelo RIPR (Reachability, Infection, Propagation, Revealability).
- Patrón Arrange–Act–Assert (AAA) y oráculos.
- Criterios de cobertura y *test requirements*.
- Contrato de equals/hashCode y herencia.

## Preguntas y respuestas

### Pregunta 1

Defina los conceptos de *fault*, *error* y *failure*. ¿Por qué es importante distinguirlos?

### Respuesta

Un *fault* (defecto) es una construcción incorrecta en el código fuente, especificación o diseño: existe estáticamente desde que el programador la escribió. Un *error* es un estado interno incorrecto del programa en tiempo de ejecución (una variable mal valuada) producido al ejecutar el *fault*. Un *failure* es la manifestación externa observable de ese error: una salida desviada de la especificación. La distinción es esencial porque un *fault* puede existir sin manifestarse: si no se ejecuta, o no infecta el estado, o no se propaga a la salida, nunca habrá *failure*. El objetivo del testing es *provocar failures* para luego diagnosticar *faults*.

### Referencia y ejemplo

Ammann & Offutt, *Introduction to Software Testing*, Cap. 1 (Def. 1.1–1.3); notas-00 y notas-01 de la cátedra. *Ejemplo*: en `return a/b`; sin chequear `b`, el *fault* es la falta de validación, el *error* es la división por cero al ejecutarse con `b=0`, y el *failure* es la excepción `ArithmeticException` visible al cliente.

### Pregunta 2

Distinga *verificación* y *validación*. ¿Por qué se las menciona juntas como V&V?

### Respuesta

La *verificación* responde a “¿construimos el producto correctamente?”: comprueba que la implementación cumpla la especificación. La *validación* responde a “¿construimos el producto correcto?”: evalúa si la especificación realmente refleja las necesidades del usuario. Se mencionan juntas porque un sistema puede pasar la verificación (cumple su spec) y fallar la validación (la spec era incorrecta), o viceversa. Ambas son necesarias para garantizar calidad.

### Referencia y ejemplo

Ammann & Offutt, Cap. 1 (Def. 1.4 y 1.5); notas-00. *Ejemplo*: si la spec exige redondeo bancario y el código lo implementa exactamente, verifica bien; pero si los contadores esperaban redondeo financiero distinto, la validación falla.

### Pregunta 3

Enuncie las cuatro condiciones del modelo RIPR y explique por qué un test puede ejecutar código defectuoso sin detectarlo.

### Respuesta

RIPR es el conjunto de condiciones necesarias para que un *fault* sea detectado: **R– Reachability**: el camino de ejecución alcanza la sentencia defectuosa. **I– Infection**: al ejecutarse, el estado interno difiere del que tendría el programa correcto. **P– Propagation**: ese estado infectado se propaga hasta una salida observable. **R– Revealability**: el oráculo detecta la diferencia y falla el test. Un test puede ejecutar el defecto y pasar si alguna condición no se cumple, p. ej. si infecta pero el estado se sobrescribe antes de propagarse.

### Referencia y ejemplo

Ammann & Offutt, Cap. 2; notas-02. *Ejemplo*: una asignación errónea `x=x+1` en lugar de `x=x-1` dentro de una rama que luego sólo retorna `true/false` no propaga el valor de `x`, por lo que el bug queda oculto.

### Pregunta 4

¿Por qué el *testing exhaustivo* es impracticable y qué implica esto para la disciplina?

### Respuesta

El espacio de entradas posibles crece típicamente de forma combinatoria (o es infinito, p. ej. enteros, strings), de modo que probar todos los casos no es viable en tiempo finito. La consecuencia es que el testing nunca demuestra ausencia de defectos: solo evidencia su presencia (Dijkstra). Por eso se trabaja con *criterios de cobertura* que seleccionan un subconjunto razonablemente pequeño y diverso de entradas con buena capacidad de revelar fallas.

### Referencia y ejemplo

Ammann & Offutt, Cap. 1; notas-01. *Ejemplo*: una función `int sumar(int, int)` tiene  $2^{64}$  pares de entradas; en lugar de probarlos todos se eligen casos de equivalencia (positivos, negativos, ceros) y bordes (`Integer.MAX_VALUE`).

### Pregunta 5

¿Qué son los *niveles* de testing (unitario, integración, sistema, aceptación) y qué pregunta intenta responder cada uno?

### Respuesta

**Unitario**: se prueban componentes aislados (clases, métodos). Pregunta: “¿El componente cumple su contrato?”. **Integración**: se prueba la interacción entre componentes. Pregunta: “¿Se comunican correctamente?”. **Sistema**: se prueba la aplicación completa contra requisitos funcionales/no funcionales. Pregunta: “¿El sistema cumple su spec global?”. **Aceptación**: el cliente o usuario verifica que satisface sus necesidades. Pregunta: “¿Es lo que se necesitaba?”. Cada nivel ataca defectos distintos: la mayoría de los *faults* se detecta más barato en unitario; los de protocolo, en integración.

### Referencia y ejemplo

Ammann & Offutt, Cap. 1 (sec. *Testing levels*); notas-00. *Ejemplo*: un test unitario verifica `Stack.push`; uno de integración, que `StackService` usa correctamente la `Stack`; uno de sistema, el flujo completo desde la API; el de aceptación, que el caso de uso del cliente funciona.

### Pregunta 6

¿Qué es el patrón Arrange-Act-Assert (AAA) y por qué se considera buena práctica?

### Respuesta

AAA estructura cada caso en tres bloques: **Arrange** (preparación del escenario: objetos, datos, fixtures), **Act** (*una sola* invocación a la unidad bajo prueba) y **Assert** (verificación del resultado con `assertEquals`, `assertTrue`, etc.). Se recomienda porque hace al test legible, independiente y mantenible. Evita tests con múltiples acciones encadenadas en los que, al fallar, no queda claro cuál operación rompió.

**Referencia y ejemplo**

Notas-01 (*patrón AAA*); convención difundida en la comunidad xUnit. *Ejemplo:* `Stack s = new Stack(); (Arrange) s.push(7); (Act) assertEquals(7, s.top()); (Assert).`

**Pregunta 7**

¿Qué es un *oráculo* de test y qué tipos existen?

**Respuesta**

Un *oráculo* es el mecanismo que decide si una ejecución es correcta o incorrecta. Tipos clásicos: **oráculo de salida esperada** (se compara el resultado con un valor predefinido); **oráculo derivado** (otra implementación de referencia o “*golden*”); **oráculo de invariantes** (`repOK`, postcondiciones); **metamórfico** (relación entre dos ejecuciones); **humano** (inspección visual). Sin oráculo, un test no decide nada: solo ejecuta.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 1; notas-01. *Ejemplo:* para `sort`, el oráculo puede ser “el resultado es una permutación ordenada de la entrada” (invariante) o “coincide con `Collections.sort`” (referencia).

**Pregunta 8**

¿Qué es un *criterio de cobertura* y qué se entiende por *test requirement*?

**Respuesta**

Un *criterio de cobertura* es una regla sistemática para derivar un conjunto de obligaciones de prueba (*test requirements*) a partir de un artefacto del sistema (código, especificación, modelo). Cada *test requirement* es una condición específica que algún test de la suite debe satisfacer (p. ej. “ejecutar la arista  $e_3$ ”). La cobertura de la suite es la fracción de *test requirements* cumplidos. Los criterios formalizan “cuántos tests son suficientes” y permiten comparar suites objetivamente.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 2 (Def. 2.6 y 2.7); notas-02. *Ejemplo:* el criterio *Statement Coverage* genera un requirement por cada sentencia del código; el criterio queda 100 % cubierto cuando la suite ejecuta todas.

**Pregunta 9**

¿Por qué la herencia que extiende una clase concreta y redefine `equals` para agregar un nuevo aspecto rompe el contrato del método?

**Respuesta**

Si la subclase considera un campo extra en `equals` pero la superclase no, se viola *simetría*: `padre.equals(hijo)` puede ser `true` (la superclase ignora el nuevo campo) mientras que `hijo.equals(padre)` es `false` (la subclase exige instancia de subclase). Si se relaja con `instanceof`, se pierde *transitividad*. Bloch concluye que no hay solución compatible con el contrato; la alternativa correcta es composición en vez de herencia.

**Referencia y ejemplo**

J. Bloch, *Effective Java*, ítem “Obey the general contract when overriding equals”; notas-01. *Ejemplo*: `ColorPoint` extiende `Point` con `equals` que compara color; `new Point(1,1).equals(new ColorPoint(1,1,RED))` es `true` pero la inversa es `false`.

**Pregunta 10**

¿Cuál es la relación obligatoria entre `equals` y `hashCode`, y qué ocurre cuando se viola?

**Respuesta**

El contrato exige: si `a.equals(b)` entonces `a.hashCode() == b.hashCode()` (la recíproca no es obligatoria). Si se viola, las colecciones basadas en *hashing* (`HashSet`, `HashMap`, `Hashtable`) fallan silenciosamente: pueden no encontrar un elemento previamente insertado (lo buscan en otro *bucket*) o admitir duplicados lógicamente iguales.

**Referencia y ejemplo**

API de `java.lang.Object` (JDK); Bloch, *Effective Java*, ítem “Always override hashCode when you override equals”; notas-01. *Ejemplo*: si `Point.equals` compara coordenadas pero `hashCode` usa la identidad por defecto, dos puntos (1,1) insertados en `HashSet` son tratados como distintos.

## 2. Automatización con JUnit, data-driven testing y repOK

### Temas abordados

- Automatización de pruebas y herramientas de *build*.
- Tests parametrizados (`@ParameterizedTest`) y fuentes de datos.
- Fixtures e independencia entre tests.
- Invariante de representación y `repOK` como oráculo interno.
- Reproducibilidad y test *flaky*.

### Preguntas y respuestas

**Pregunta 11**

¿Qué se entiende por *automatización* de pruebas y por qué es relevante?

**Respuesta**

Automatizar pruebas significa hacer que la ejecución y verificación de los tests las realice una herramienta sin intervención humana, integrada al proceso de *build*. Es relevante porque permite ejecutar la suite a cada cambio (*continuous integration*), detecta regresiones temprano, libera al programador de tareas repetitivas y hace que los resultados sean reproducibles. Sin automatización, los tests pierden frescura y dejan de ejecutarse.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 3; notas-03 (*automation*). *Ejemplo*: ejecutar `mvn test` compila el código, corre todos los `@Test` de JUnit y reporta éxitos/fallas en formato uniforme.

**Pregunta 12**

¿Qué es un test *data-driven* y qué ventajas tiene sobre escribir un método por cada caso?

**Respuesta**

En un test *data-driven* la lógica de verificación se escribe una sola vez y los casos se alimentan como filas de datos. En JUnit 5 se implementa con `@ParameterizedTest` y una fuente de datos. Ventajas: reduce la duplicación, facilita agregar nuevos casos (sólo una fila), separa datos de código y mejora la legibilidad del informe (cada fila aparece como test independiente con su propio nombre).

**Referencia y ejemplo**

Notas-04 (*data-driven test*); guía de JUnit 5 (*Parameterized Tests*). *Ejemplo*: un único método `esPrimo(int n, boolean esp)` con `@CsvSource({"2,true","4,false","17,true"})` reemplaza tres tests separados.

**Pregunta 13**

Compare `@CsvSource`, `@MethodSource` y `@CsvFileSource`: ¿cuándo conviene cada una?

**Respuesta**

**@CsvSource**: para pocos casos simples; los datos van *inline* como CSV. La más rápida. **@MethodSource**: apunta a un método estático que retorna `Stream<Arguments>`; conviene cuando los argumentos son objetos complejos o requieren construcción programática. **@CsvFileSource**: lee los datos desde un archivo CSV externo en `src/test/resources`; útil con muchos casos o cuando los datos se mantienen aparte del código por motivos de mantenimiento.

**Referencia y ejemplo**

Notas-04; documentación oficial de JUnit 5. *Ejemplo*: `@CsvFileSource(resources="/casosBisiestos.csv")` alimenta cientos de pares (año, esperado) producidos por una planilla.

**Pregunta 14**

¿Qué son las *fixtures* y qué papel cumplen las anotaciones `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`?

**Respuesta**

Una *fixture* es el estado preparado que necesitan los tests para correr (objetos, conexiones, datos de prueba). `@BeforeEach/@AfterEach` corren antes/después de *cada* test y sirven para construir/limpiar estado local. `@BeforeAll/@AfterAll` corren una vez para *toda* la clase (deben ser estáticos) y se usan para recursos caros (abrir/cerrar BD, levantar servidor). Estructuran el Arrange y aseguran independencia entre tests.

**Referencia y ejemplo**

Notas-03; guía de JUnit 5 (*Lifecycle annotations*). *Ejemplo*: `@BeforeEach void init(){ stack = new Stack(); }` garantiza una pila vacía en cada test.

**Pregunta 15**

¿Qué es el invariante de representación y para qué sirve el método `repOK()`?

**Respuesta**

El invariante de representación es la propiedad que debe cumplir el estado interno de una clase para representar un valor válido del TAD que implementa. `repOK()` retorna `true` si el objeto satisface su invariante. Se usa como *oráculo interno*: tras cada operación se invoca para detectar estados corruptos que el resultado externo aún no manifiesta. Encuentra fallas que la verificación de salida no detecta y refuerza cualquier oráculo.

**Referencia y ejemplo**

B. Liskov & J. Guttag, *Program Development in Java*, cap. 5; notas-03. *Ejemplo*: en una cola circular de capacidad  $C$ , `repOK()` chequea  $0 \leq \text{size} \leq C$  y `back == (front + size) % C`.

**Pregunta 16**

¿Por qué los tests deben ser *independientes* entre sí?

**Respuesta**

Si un test depende del estado dejado por otro, el orden de ejecución condiciona el resultado: un test podría pasar aislado y fallar en suite, o viceversa. La independencia se logra reiniciando fixtures (`@BeforeEach`), evitando estado estático compartido y no asumiendo orden. Permite paralelización, debugging local y ejecución parcial sin sorpresas.

**Referencia y ejemplo**

Meszaros, *xUnit Test Patterns (Independent Tests)*; notas-03. *Ejemplo*: dos tests que comparten un `static Stack` pueden interferir; reemplazarlo por una instancia inicializada en `@BeforeEach` elimina el acoplamiento.

**Pregunta 17**

¿Qué hace que un test sea *reproducible* y qué es un test *flaky*?

**Respuesta**

Un test es *reproducible* cuando su resultado depende solo de su entrada controlada: dado el mismo código y datos, siempre pasa o siempre falla. Es *flaky* cuando exhibe ambos resultados sin cambios reales, típicamente por dependencia de tiempo, ordenamiento, concurrencia, red, aleatoriedad no controlada o estado externo. Los *flaky* erosionan la confianza en la suite: se atribuye una falla real a flakiness y se ignora. La solución es controlar la fuente del no-determinismo (semilla fija, reloj inyectado, mock de red).

**Referencia y ejemplo**

Notas-03; literatura de *flaky tests* (Luo et al., 2014). *Ejemplo*: `assertTrue(Math.random() < 0.5)` es trivialmente *flaky*; `new Random(42).nextDouble()` lo hace determinista.

**Pregunta 18**

¿Qué buenas prácticas se siguen al nombrar y organizar tests?

**Respuesta**

Cada test debe leerse como un enunciado: el nombre describe el escenario y el comportamiento esperado (p. ej. `pop_sobre_pila_vacia_lanza_excepcion`). Se agrupan en clases por unidad bajo prueba, separadas físicamente del código productivo (`src/test/java`). Conviene un test por concepto, sin lógica complicada y sin múltiples “act” encadenados. Buenos nombres sirven



como documentación viva del contrato.

#### Referencia y ejemplo

Notas-03; convenciones de la comunidad xUnit. *Ejemplo:* `addDays_a_31_de_enero_devuelve_1_de_febrero` se entiende sin leer el cuerpo.

#### Pregunta 19

¿Cómo se aborda el *debugging sistemático* de un *fault* detectado por un test?

#### Respuesta

Una vez que un test falla, el flujo recomendado es: **(1)** reproducir la falla con un test mínimo y determinista; **(2)** aplicar bisección (en código: `git bisect`; en datos: reducir la entrada hasta el caso mínimo) para localizar el punto de regresión o el subcaso responsable; **(3)** formular una hipótesis sobre la causa, validarla con un test adicional o con un *print*/depurador; **(4)** corregir el *fault* y dejar el test mínimo en la suite como prueba de no regresión.

#### Referencia y ejemplo

Zeller, *Why Programs Fail*; notas-03. *Ejemplo:* ante una falla del cálculo del año bisiesto, reducir el caso “año=2008, días=366” hasta aislar el bucle, corregir la guarda y dejar ese caso parametrizado en la suite.

#### Pregunta 20

¿Cuál es el rol de Maven (u otra herramienta de *build*) en un proyecto de testing?

#### Respuesta

Maven estandariza el ciclo *build/test/package*, gestiona dependencias declaradas en el `pom.xml`, fija versiones de JUnit/JaCoCo/Pitest, ejecuta la suite (`mvn test`) y produce reportes en formato uniforme. Hace que el proyecto se construya igual en cualquier máquina y se integre a CI. Sin un *build* declarativo, los tests pueden funcionar localmente y romperse en otra máquina por configuración divergente.

#### Referencia y ejemplo

Documentación de Apache Maven; notas-03. *Ejemplo:* agregar JUnit 5 implica añadir la dependencia `junit-jupiter` al `pom.xml`; cualquier desarrollador que clone el repo y corra `mvn test` reproduce la suite.

## 3. Input Space Partitioning (ISP)

### Temas abordados

- Modelo del Dominio de Entradas (MDE): características, particiones y bloques.
- Particionamiento por equivalencia y *boundary value analysis*.
- Criterios combinatorios: Each Choice, Pair-Wise, Base Choice, All Combinations.
- Restricciones de factibilidad.
- Subsunción y costo de los criterios.



## Preguntas y respuestas

### Pregunta 21

¿Qué es el testing funcional (caja negra) y cómo se ubica ISP dentro de él?

### Respuesta

El testing funcional (o caja negra) deriva los tests exclusivamente de la *especificación*, sin mirar la implementación: se trata el sistema como una caja con entradas y salidas. ISP es la técnica funcional canónica: parte del dominio de entradas, lo divide en bloques de equivalencia y construye casos combinando bloques. Su fuerza es la independencia del código y su debilidad es no garantizar cobertura estructural.

### Referencia y ejemplo

Ammann & Offutt, Cap. 6; notas-05 (*criterios*). *Ejemplo*: probar una API REST sólo con `curl` contra los endpoints documentados es testing funcional puro.

### Pregunta 22

¿Qué es un Modelo del Dominio de Entradas (MDE) y cuáles son sus elementos?

### Respuesta

El MDE abstrae el dominio de entradas de la unidad bajo prueba a partir de su especificación. Sus elementos son: **Parámetros** (variables de entrada: argumentos, estado relevante, entorno); **Características** (propiedades semánticas relevantes de cada parámetro, p. ej. “signo de  $x$ ”); **Particiones (bloques)** (subconjuntos disjuntos y exhaustivos de valores por cada característica); **Restricciones de factibilidad** (combinaciones inválidas según la semántica).

### Referencia y ejemplo

Ammann & Offutt, Cap. 6 (Def. 6.1–6.4); notas-06 (*input*). *Ejemplo*: para `indexOf(lista, item)`, una característica es “presencia” con bloques {no aparece, aparece una vez, aparece más de una vez}.

### Pregunta 23

Distinga las características de *interface* y de *functionality*. ¿Cuál es más informativa?

### Respuesta

Las características *interface-based* se derivan exclusivamente de la firma del método (tipo de parámetros: nulo/no nulo, número/string vacío, etc.). Las *functionality-based* se derivan de lo que la unidad *hace* según su spec (relaciones entre parámetros, estados relevantes, casos especiales del dominio). Las *functionality-based* son más informativas porque capturan lógica del problema, pero exigen entender la especificación; las *interface-based* son más mecánicas y rápidas, pero más débiles.

### Referencia y ejemplo

Ammann & Offutt, Cap. 6 (sec. *Interface vs Functionality*); notas-06. *Ejemplo*: para `busqueda(arr, x)`, “ $x$  es null” es *interface*; “ $x$  aparece en la primera mitad de `arr`” es *functionality*.

**Pregunta 24**

¿Qué propiedades deben cumplir los bloques de una partición y por qué?

**Respuesta**

Los bloques deben ser: **disjuntos** (sin solaparse: un mismo valor no pertenece a dos bloques) y **completos/exhaustivos** (su unión cubre todo el dominio de la característica). Esto garantiza que cada valor concreto del dominio pertenezca a exactamente un bloque, condición necesaria para que los criterios combinatorios derivados estén bien definidos.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6 (Def. 6.5); notas-06. *Ejemplo*: para “signo de  $x$ ”, los bloques {negativo, cero, positivo} son disjuntos y completos;  $\{\leq 0, \geq 0\}$  se solaparía en  $x = 0$ .

**Pregunta 25**

¿Qué relación hay entre el *equivalence partitioning* clásico, el *boundary value analysis* (BVA) e ISP?

**Respuesta**

El *equivalence partitioning* clásico es la idea básica: dividir el dominio en clases de equivalencia y testear un representante por clase, asumiendo comportamiento uniforme dentro de cada clase. *BVA* complementa eligiendo valores en los *bordes* de las clases (donde típicamente surgen los *faults*: comparaciones  $<$  vs  $<=$ ). *ISP* generaliza esto a múltiples características simultáneas, agregando los criterios combinatorios para decidir cómo cubrir las interacciones.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6; G. Myers, *The Art of Software Testing*; notas-06. *Ejemplo*: si una función admite 1..100, *BVA* prueba 0, 1, 100, 101; *ISP* combina ese rango con otras características como “signo” o “paridad”.

**Pregunta 26**

Compare los criterios Each Choice (EC), Pair-Wise (PWC), Base Choice (BCC) y All Combinations (ACC).

**Respuesta**

**Each Choice**: cada bloque de cada característica aparece en al menos un test. Mínimo de tests, cobertura más débil. **Pair-Wise**: para cada par de características, todos los pares de bloques aparecen juntos en al menos un test. Detecta interacciones entre pares. **Base Choice**: se elige un valor base por característica; un test usa todos los base, y luego se genera un test por cada otro bloque variando *una sola* característica. Ideal cuando hay un escenario “normal”. **All Combinations**: producto cartesiano de bloques; cobertura máxima pero explosión combinatoria. La cantidad de tests crece:  $EC \leq PWC, BCC \leq ACC$ .

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6 (sec. *Coverage criteria for ISP*); notas-06. *Ejemplo*: con 3 características de 3 bloques cada una, *EC* necesita 3 tests, *ACC* necesita 27.

**Pregunta 27**

¿Qué papel cumplen las restricciones de factibilidad al aplicar un criterio combinatorio?

**Respuesta**

Marcan combinaciones de bloques inválidas según la spec (p. ej. “lista **null**” incompatible con “primer elemento es entero”). Al aplicar PWC o ACC, primero se generan formalmente todas las combinaciones y luego se descartan las infactibles, o se reemplazan por un caso equivalente que cumpla la restricción. Sin esta etapa, los requirements incluirían tests imposibles de instanciar, inflando la cuenta y bajando la efectividad real.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6; notas-06. *Ejemplo*: en `contains(lista, x)`, “lista vacía” es incompatible con “*x* aparece más de una vez”; esa combinación se descarta o se sustituye por la más cercana factible.

**Pregunta 28**

Esboce el orden de subsunción entre EC, PWC, BCC y ACC.

**Respuesta**

Sobre el mismo MDE: **ACC subsume a PWC y a EC** (cubrir todas las combinaciones implica cubrir todos los pares y todos los bloques). **PWC subsume a EC** (cubrir pares cubre bloques individuales). **BCC subsume a EC** pero no es comparable con PWC o ACC en general (BCC cubre todos los bloques pero no necesariamente todos los pares). En la práctica, el orden cubierto  $\rightarrow$  caro es:  $EC \rightarrow BCC/PWC \rightarrow ACC$ .

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6; notas-06. *Ejemplo*: una suite ACC con 27 tests automáticamente cumple PWC; una suite EC de 3 tests no necesariamente cubre los pares.

**Pregunta 29**

¿Cómo se construye un MDE a partir de una especificación dada?

**Respuesta**

Pasos típicos: **(1)** listar parámetros de entrada (argumentos, estado relevante, entorno); **(2)** para cada parámetro, identificar características semánticas (qué propiedades cambian el comportamiento); **(3)** para cada característica, definir bloques de equivalencia disjuntos y completos, incluyendo bordes; **(4)** listar restricciones de factibilidad; **(5)** elegir un criterio combinatorio acorde a costo/coertura y derivar los requirements.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 6 (sec. *Designing input space partitions*); notas-06. *Ejemplo*: para `descuento(precio, cliente)` se identifican `precio` (negativo/cero/positivo) y `cliente` (anónimo, registrado, VIP) y se restringe “precio negativo  $\Rightarrow$  excepción”.

**Pregunta 30**

¿Cómo se decide qué criterio combinatorio usar en un proyecto real?

**Respuesta**

Se equilibran cobertura y costo. Si hay pocos casos y se busca rapidez, EC. Si hay un escenario “normal” bien definido y se quiere aislar desviaciones, BCC. Si los defectos esperables surgen de interacciones, PWC. Si el costo de un *failure* es altísimo (avión, salud) y el dominio es chico,

ACC. Conviene declarar explícitamente requirements inviables para no inflar la suite con casos imposibles.

#### Referencia y ejemplo

Ammann & Offutt, Cap. 6; D. Kuhn et al., NIST sobre PWC. *Ejemplo*: estudios empíricos del NIST muestran que el 70–90 % de los *faults* en software comercial se activa con interacciones de a lo sumo dos parámetros, lo que justifica PWC como punto medio.

## 4. Testing basado en grafos

### Temas abordados

- Grafos de Flujo de Control (CFG): nodos, aristas, caminos.
- Criterios: *Node*, *Edge*, *Edge-Pair*, *Prime Path*.
- *Best Effort Touring*: *sidetrips* y *detours*.
- Subsunción entre criterios.
- Complejidad ciclomática e infactibilidad de caminos.

### Preguntas y respuestas

#### Pregunta 31

¿Cómo se construye un Grafo de Flujo de Control (CFG) a partir del código fuente?

#### Respuesta

Se identifican *bloques básicos*: secuencias máximas de instrucciones sin saltos ni puntos de entrada intermedios. Cada bloque es un nodo. Se agrega una arista de  $n_i$  a  $n_j$  cuando el control puede pasar inmediatamente entre ellos (caída, salto condicional, salto incondicional, retorno). Las decisiones generan nodos con dos aristas de salida; las uniones, nodos con varias aristas de entrada. Se añaden un nodo inicial y nodos finales explícitos.

#### Referencia y ejemplo

Ammann & Offutt, Cap. 7; notas-08 (*graph from source code*). *Ejemplo*: `if(x>0) y=1; else y=-1;` produce un nodo con la condición, dos hijos (una por rama) y un nodo de unión donde el flujo se reúne.

#### Pregunta 32

Defina los criterios *Node Coverage* (NC), *Edge Coverage* (EC) y *Edge-Pair Coverage* (EPC).

#### Respuesta

**NC**: todo nodo del CFG debe ser visitado por al menos un test. **EC**: toda arista debe ser recorrida por al menos un test. **EPC**: toda secuencia de dos aristas consecutivas (par de aristas que comparten un nodo intermedio) debe ser recorrida. La fortaleza crece:  $NC \subseteq EC \subseteq EPC$ , porque recorrer aristas implica visitar los nodos, y recorrer pares implica recorrer aristas.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (Def. 7.1–7.5); notas-07. *Ejemplo:* en un **if-else**, NC se cubre con un único test que pase por ambas ramas en ejecuciones distintas; EC también; EPC obliga, en un loop, a cubrir “entrar y salir” y “entrar y volver a entrar”.

**Pregunta 33**

¿Por qué EC subsume a NC?

**Respuesta**

Para recorrer una arista hay que entrar y salir de sus nodos extremos: cualquier suite que recorra todas las aristas necesariamente visita todos los nodos involucrados. Los nodos aislados se incluyen como aristas hacia/desde un nodo inicial/final, de modo que la cobertura de aristas implica cobertura de nodos.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (Subsumption hierarchy); notas-07. *Ejemplo:* en un grafo de 3 nodos  $A \rightarrow B \rightarrow C$ , cubrir las dos aristas  $A \rightarrow B$  y  $B \rightarrow C$  automáticamente visita  $A$ ,  $B$  y  $C$ .

**Pregunta 34**

¿Qué es un *simple path* y un *prime path*? ¿Qué exige el criterio PPC?

**Respuesta**

Un *simple path* es un camino sin nodos repetidos, salvo posiblemente el primero y el último que coincidan (ciclo simple). Un *prime path* es un *simple path maximal*: no es subpath propio de otro simple path. PPC (*Prime Path Coverage*) exige cubrir todos los prime paths del grafo. Es un criterio fuerte porque garantiza explorar todos los ciclos básicos y todas las combinaciones maximales de ramas.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (Def. 7.10 y 7.11); notas-07. *Ejemplo:* en un loop **while(c) S**; los prime paths incluyen “no entrar al loop”, “entrar y salir tras una iteración” y “ciclar al menos dos veces”.

**Pregunta 35**

Explique las nociones de *tour*, *sidetrip* y *detour*, y por qué se introducen.

**Respuesta**

Un test “cubre” (*tours*) un camino requerido si lo recorre como subpath de su ejecución. Algunos caminos requeridos son infactibles directamente (condiciones contradictorias). Se permiten relajaciones: **sidetrip**, desvíos que abandonan y vuelven al camino *por el mismo nodo*; **detour**, desvíos que reingresan por otro nodo posterior. Esta política se llama *Best Effort Touring* y permite cubrir prime paths que en forma estricta serían infactibles.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (Def. 7.6–7.8); notas-07. *Ejemplo:* si un prime path exige ir  $A \rightarrow B \rightarrow D$  pero  $B$  siempre fuerza pasar por  $C$ , un *sidetrip* por  $C$  permite tour  $A \rightarrow B \rightarrow C \rightarrow B \rightarrow D$  (en grafos con auto-retorno) o aceptar el desvío equivalente.

**Pregunta 36**

¿Qué es la *complejidad ciclomática* de McCabe y cómo se relaciona con el testing?

**Respuesta**

La complejidad ciclomática  $V(G)$  de un CFG es el número de caminos linealmente independientes; se calcula como  $V(G) = E - N + 2P$  (aristas, nodos, componentes conexas) o, equivalentemente, como el número de decisiones más uno. Es una cota inferior sugerida para el número de tests que cubran cada rama: una suite con  $V(G)$  caminos bien elegidos garantiza EC. También se usa como métrica de complejidad/mantenibilidad: módulos con  $V(G) > 10$  se consideran difíciles de testear.

**Referencia y ejemplo**

T. McCabe, *A Complexity Measure*, 1976; notas-07. *Ejemplo*: un método con tres `if` secuenciales tiene  $V(G) = 4$  y se recomienda al menos 4 tests para cubrir todas las ramas.

**Pregunta 37**

¿Qué se entiende por *infactibilidad* de un camino o requirement y cómo se trata?

**Respuesta**

Un camino es *infactible* si no existe ninguna entrada que lo recorra (condiciones contradictorias o estados imposibles). Determinar infactibilidad es indecidible en general. En la práctica, se inspecciona manualmente, se descarta el requirement como infactible y se documenta la razón, o se relaja con *sidetrip/detour*. No se debe “forzar” la cobertura con tests artificiales que no respeten la semántica.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (sec. *Infeasible requirements*); notas-07. *Ejemplo*: un prime path exige `x>0 && x<0` en sentencias sucesivas sin modificación intermedia: infactible.

**Pregunta 38**

Esboce el orden de subsunción entre los criterios basados en grafos.

**Respuesta**

Sobre el mismo CFG, asumiendo *best effort touring*:  $\text{Complete Path Coverage} \supseteq \text{Prime Path Coverage} \supseteq \text{Edge-Pair Coverage} \supseteq \text{Edge Coverage} \supseteq \text{Node Coverage}$ . CPC suele ser inalcanzable (infinitos caminos con loops), por eso PPC es el más fuerte que se exige en la práctica.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (Subsumption hierarchy); notas-07. *Ejemplo*: una suite PPC sobre una función con loops automáticamente satisface EPC, EC y NC.

**Pregunta 39**

Más allá del CFG, ¿qué otros grafos se usan en testing y para qué?

**Respuesta**

**Grafos de llamadas** (*call graphs*): nodos son métodos, aristas indican invocación; se aplican criterios análogos al CFG para cobertura entre métodos en testing de integración. **Autómatas finitos** / **FSM**: modelan estados y transiciones del sistema; se aplica NC/EC sobre el modelo

(testing basado en modelos). **Grafos de uso-definición** (*def-use*): vinculan definiciones de variables con sus usos para *data-flow coverage*.

#### Referencia y ejemplo

Ammann & Offutt, Cap. 7 (*Other graph coverage criteria*); notas-07. *Ejemplo*: para una clase `LoginManager` modelada como FSM con estados {anónimo, autenticado, bloqueado}, EC sobre el grafo asegura ejercitar cada transición.

#### Pregunta 40

¿Qué es la *instrumentación* de código y para qué se usa en cobertura basada en grafos?

#### Respuesta

Instrumentar significa insertar trazas en cada nodo o arista del CFG (manualmente o vía un agente como JaCoCo) para que el programa registre los nodos/aristas/caminos recorridos durante la ejecución de la suite. Sirve para *medir* la cobertura real, identificar nodos/aristas no ejercitados y guiar el diseño de tests adicionales. Sin instrumentación, la cobertura es una estimación basada en el análisis manual del CFG.

#### Referencia y ejemplo

Ammann & Offutt, Cap. 7; notas-08; documentación de JaCoCo. *Ejemplo*: tras correr la suite, JaCoCo reporta “rama `else` de la línea 47 no cubierta”, pista directa para escribir el test faltante.

## 5. Cobertura lógica de predicados

### Temas abordados

- Predicados, cláusulas y tablas de verdad.
- Predicate Coverage (PC), Clause Coverage (CC), Combinatorial Coverage (CoC).
- Active Clause Coverage: GACC, CACC, RACC.
- Relación con MCDC y software crítico.
- Cobertura lógica desde código vs. desde especificación.

### Preguntas y respuestas

#### Pregunta 41

Distinga *predicado* y *cláusula*. Dé un ejemplo concreto.

#### Respuesta

Un *predicado* es una expresión booleana completa que controla una decisión (la condición de un `if`, `while`, `return`). Una *cláusula* es una sub-expresión booleana sin operadores lógicos en su raíz: típicamente una comparación atómica (`a>0`) o una variable booleana. La diferencia importa porque los criterios trabajan a uno u otro nivel.



**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Def. 8.1 y 8.2); notas-09. *Ejemplo:* en `if((a>0) && (b==c || d<5))`, el predicado es toda la condición y las cláusulas son `a>0`, `b==c` y `d<5`.

**Pregunta 42**

¿Qué exigen Predicate Coverage (PC) y Clause Coverage (CC)? ¿Cuál es más fuerte?

**Respuesta**

**PC:** para cada predicado, debe existir un test que lo haga verdadero y otro que lo haga falso.  
**CC:** para cada cláusula de cada predicado, debe existir un test que la haga verdadera y otro falsa. Ninguno subsume al otro: una suite puede cumplir CC sin cubrir ambos valores del predicado completo, y viceversa. Son criterios *incomparables*.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Def. 8.3 y 8.4); notas-09. *Ejemplo:* para  $P = A \vee B$ , los tests  $(A, B) \in \{(T, F), (F, T)\}$  cubren CC pero  $P$  siempre vale **true**, así que no cumplen PC.

**Pregunta 43**

¿Qué significa que una cláusula sea *activa* en un predicado?

**Respuesta**

Una cláusula  $c$  es *activa* en un predicado  $P$  cuando, fijado el resto, cambiar el valor de  $c$  cambia el valor de  $P$ . Equivalentemente, las demás cláusulas no “enmascaran” a  $c$ : el resultado depende efectivamente de su valor. Esta noción se calcula con la *derivada booleana* de  $P$  respecto de  $c$ : la condición  $P_c \neq P_{\neg c}$  debe ser **true**.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Def. 8.5); notas-09. *Ejemplo:* en  $P = A \wedge B$ ,  $A$  es activa cuando  $B = T$  (cambiar  $A$  cambia  $P$ ); cuando  $B = F$ ,  $P$  vale **false** sin importar  $A$ , así que  $A$  está inactiva.

**Pregunta 44**

Defina GACC, CACC y RACC y explique sus diferencias.

**Respuesta**

Las tres exigen, para cada cláusula  $c$ , dos tests donde  $c$  sea activa y tome ambos valores.  
**GACC (General):** las demás cláusulas pueden tomar *cualquier* valor en cada test, siempre que  $c$  sea activa en ambos.  
**CACC (Correlated):** se exige además que el predicado tome valores distintos entre ambos tests.  
**RACC (Restricted):** las demás cláusulas deben tener *exactamente los mismos* valores en ambos tests (solo cambia  $c$ ). Fortaleza creciente:  $GACC \subseteq CACC \subseteq RACC$ , pero RACC suele tener requirements infactibles.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Def. 8.6–8.8); notas-09 y notas-09-02. *Ejemplo:* en  $P = A \wedge (B \vee C)$ , RACC para  $A$  exige dos tests con  $B, C$  idénticos pero  $A$  distinto; GACC admite cualquier combinación que mantenga  $A$  activa.

**Pregunta 45**

¿Qué es MCDC y qué relación tiene con CACC?

**Respuesta**

MCDC (*Modified Condition/Decision Coverage*) es el criterio exigido por la norma DO-178B/C nivel A para software aeronáutico crítico: cada cláusula debe demostrarse que afecta independientemente el resultado del predicado. La formulación moderna del libro *equipara MCDC con CACC*: ambos requieren, para cada cláusula, dos tests donde ella sea activa, cambie valor y el predicado cambie valor. CACC es la versión académica/formal del concepto industrial MCDC.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8; RTCA DO-178C; notas-09. *Ejemplo*: el software de control de un avión Boeing/Airbus debe certificarse con MCDC, lo que en la práctica equivale a satisfacer CACC sobre todos los predicados.

**Pregunta 46**

¿Qué es *Combinatorial Coverage* (CoC) sobre predicados? ¿Qué problema tiene?

**Respuesta**

CoC exige cubrir *todas* las combinaciones de valores de las cláusulas de un predicado: con  $n$  cláusulas,  $2^n$  tests por predicado. Es el criterio más exigente y subsume a todos los demás (PC, CC, GACC, CACC, RACC). Su problema es la *explosión combinatoria*: en predicados con 5–10 cláusulas, el costo se vuelve prohibitivo y muchas combinaciones suelen ser infactibles (cláusulas correlacionadas).

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Def. 8.9); notas-09. *Ejemplo*: para  $P$  con 8 cláusulas, CoC pide 256 tests; CACC con 9 ya garantiza un nivel comparable de detección en la práctica.

**Pregunta 47**

Esboce el orden de subsunción entre los criterios lógicos.

**Respuesta**

$\text{CoC} \supseteq \text{RACC} \supseteq \text{CACC} \supseteq \text{GACC} \supseteq \text{CC}$ , y  $\text{CACC} \supseteq \text{PC}$ . PC y CC son *incomparables*. La elección práctica: PC es mínimo aceptable, CACC es el estándar de la industria crítica, CoC se reserva para predicados muy pequeños o seguridad extrema.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (Subsumption hierarchy); notas-09. *Ejemplo*: una suite que cumple CACC automáticamente cumple GACC, CC y PC.

**Pregunta 48**

¿Cuál es la diferencia entre cobertura lógica obtenida desde *código fuente* y desde *especificación*?

**Respuesta**

Desde el código se extraen los predicados de los `if`, `while`, etc. ya escritos: cubre la lógica *implementada*, incluyendo bugs (si el programador olvidó una cláusula, no se exigirá testearla). Desde la especificación se extraen los predicados que la spec impone (precondiciones, postcondiciones, casos): cubre la lógica *esperada*, detectando faltantes en el código. La cátedra recomienda usar ambas vistas.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (sec. *Specification-based logic coverage*); notas-09 y notas-09-02. *Ejemplo*: la spec dice “descontar si cliente es VIP O compra > \$10000”; si el código solo verifica el primer disyunto, una cobertura basada en código no detecta la cláusula faltante; basada en spec, sí.

**Pregunta 49**

¿Cómo afecta el *cortocircuito* (`&&`, `||`) a la evaluación de la cobertura lógica?

**Respuesta**

En Java, `&&` y `||` son cortocircuitados: si el primer operando determina el resultado, el segundo *no* se evalúa. Esto hace que ciertas combinaciones de valores de cláusulas sean *inalcanzables a nivel de ejecución* (la segunda cláusula nunca se evalúa). Para análisis de cobertura lógica, esto significa que algunas filas de la tabla de verdad son inaccesibles a través de la ejecución real, aunque sigan siendo conceptualmente válidas. RACC puede pedir tests infactibles por cortocircuito.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8 (*Short-circuit evaluation*); notas-09. *Ejemplo*: en `if(a!=null && a.length>0)`, si `a==null`, la segunda cláusula no se evalúa; observar “`a.length>0` con `a=null`” es imposible.

**Pregunta 50**

¿Por qué CACC es el criterio práctico recomendado en la industria?

**Respuesta**

Porque ofrece el mejor balance costo/beneficio: requiere aproximadamente  $n + 1$  tests por predicado de  $n$  cláusulas (mucho menos que CoC, que pide  $2^n$ ) y garantiza que cada cláusula tiene efecto observable sobre el resultado. Esto detecta defectos en cualquier cláusula (operador equivocado, comparación invertida, cláusula faltante o redundante). La industria aeronáutica, ferroviaria y automotriz lo adopta como MCDC.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 8; RTCA DO-178C; ISO 26262 (sector automotriz). *Ejemplo*: para un predicado de 4 cláusulas, CACC pide  $\sim 5$  tests; CoC pediría 16.

## 6. Testing de mutación y fuzzing

### Temas abordados

- Testing basado en sintaxis: operadores de mutación.

- *Mutation score*, mutantes muertos, vivos y equivalentes.
- Hipótesis del programador competente y *coupling effect*.
- Strong vs weak mutation; Pitest.
- Fuzzing: random, mutation-based, coverage-guided; *differential testing*.

## Preguntas y respuestas

### Pregunta 51

¿Qué es un *operador de mutación* y qué es el *mutation score* de una suite?

### Respuesta

Un *operador de mutación* es una regla de transformación sintáctica que produce una variante del programa (*mutante*): cambiar  $<$  por  $<=$ ,  $+$  por  $-$ , invertir una condición, eliminar una sentencia, alterar una constante, etc. El *mutation score* de una suite es la proporción de mutantes que “mata” (al menos un test falla sobre el mutante) respecto del total de mutantes no equivalentes:

$$\text{score} = \frac{\text{mutantes muertos}}{\text{mutantes totales} - \text{mutantes equivalentes}}.$$

Es un proxy de la *fortaleza de los oráculos* de la suite, no solo su cobertura.

### Referencia y ejemplo

Ammann & Offutt, Cap. 9; notas-11 (*mutation*). *Ejemplo*: si Pitest genera 100 mutantes (3 equivalentes) y la suite mata 90, el score es  $90/97 \approx 92,8\%$ .

### Pregunta 52

Enuncie la *hipótesis del programador competente* y el *coupling effect*, y por qué fundamentan la mutación.

### Respuesta

**Programador competente**: los programadores escriben programas casi correctos; los *faults* reales son cambios pequeños respecto del programa correcto (cambiar un operador, un índice, etc.). **Coupling effect**: si una suite detecta defectos simples (mutaciones de un solo operador), también detecta defectos complejos (combinaciones de varios), porque las fallas complejas suelen “activarse” por los mismos casos que las simples. Juntos justifican usar mutantes simples como sustitutos representativos de defectos reales.

### Referencia y ejemplo

DeMillo, Lipton & Sayward (1978); Ammann & Offutt, Cap. 9; notas-11. *Ejemplo*: una suite que mata el mutante  $< \rightarrow <=$  casi siempre también detecta un bug compuesto que combina ese mismo cambio con otro.

### Pregunta 53

¿Qué es un *mutante equivalente* y por qué es problemático?

### Respuesta

Es un mutante *sintácticamente* distinto pero *semánticamente* igual al original: ninguna entrada distingue uno del otro, así que ningún test puede matarlo. Es problemático porque infla el nu-

merador de los mutantes vivos sin que la suite tenga forma de mejorarlo. Detectar equivalencia es indecible (se reduce a equivalencia de programas), así que en la práctica se inspecciona manualmente y se descuenta del denominador.

### Referencia y ejemplo

Ammann & Offutt, Cap. 9 (sec. *Equivalent mutants*); notas-11. *Ejemplo*:  $x = y + 0$ ; mutado a  $x = y$ ; es semánticamente equivalente sobre cualquier entrada.

### Pregunta 54

Distinga *strong mutation* y *weak mutation*.

### Respuesta

**Strong mutation**: un mutante se considera muerto si la suite produce una salida observable distinta de la del original (se exige propagación completa: condición P de RIPR). **Weak mutation**: basta con que el estado interno difiera inmediatamente después de ejecutar el punto mutado (basta infección, sin exigir propagación). Weak es más barato de medir (no requiere ejecutar el resto del programa) y menos exigente. La mayoría de las herramientas modernas (Pitest, MuJava) implementan strong.

### Referencia y ejemplo

Ammann & Offutt, Cap. 9 (Def. 9.4 y 9.5); notas-11. *Ejemplo*: si la mutación corrompe  $x$  pero  $x$  se sobrescribe antes de salir, strong la deja viva; weak la mata.

### Pregunta 55

¿Cómo se interpreta un reporte de Pitest y qué información provee?

### Respuesta

Pitest genera mutantes según un conjunto de operadores, ejecuta la suite JUnit sobre cada uno y reporta su estado: **KILLED** (algún test falló), **SURVIVED** (todos pasaron), **NO\_COVERAGE** (la línea no se ejecutó), **TIMED\_OUT** (loop infinito), **MEMORY\_ERROR**. El reporte HTML muestra, por línea, qué mutantes se generaron y su estado. Calcula *line coverage*, *mutation score* y *test strength* (matados / cubiertos). Identifica líneas no testeadas o débilmente testeadas.

### Referencia y ejemplo

Documentación de Pitest (<https://pitest.org>); notas-11. *Ejemplo*: un mutante **SURVIVED** en la línea 47 indica un oráculo débil; se agrega un assert que distinga el comportamiento esperado del mutado.

### Pregunta 56

¿Qué estrategia se usa para incrementar el *mutation score* de una suite?

### Respuesta

Se trabaja iterativamente: (1) se corre Pitest, (2) se inspeccionan los mutantes sobrevivientes, (3) por cada uno se determina si es equivalente (se descarta) o si se puede matar con un test específico, (4) se agrega ese test y se repite hasta alcanzar la meta o agotar mutantes no equivalentes. Los mutantes en condiciones de borde suelen requerir tests con valores extremos.

### Referencia y ejemplo

Ammann & Offutt, Cap. 9; notas-11. *Ejemplo*: el mutante  $a+b == c \rightarrow a+b == c+1$  se mata con un test exactamente en el borde de la igualdad.

### Pregunta 57

¿Qué es el *fuzzing* y qué tipos principales existen?

### Respuesta

El *fuzzing* es la técnica de generar entradas (típicamente masivas y aleatorias) para sondear el comportamiento de un programa, buscando crashes, excepciones o violaciones de invariantes. Tipos: **Random fuzzing**: entradas aleatorias dentro del dominio, sin guía. **Mutation-based fuzzing**: parte de un *seed corpus* de entradas válidas y aplica pequeñas mutaciones (cambiar bytes, duplicar, eliminar). **Generational fuzzing**: usa una gramática para generar entradas estructuradas válidas. **Coverage-guided fuzzing** (AFL, libFuzzer): instrumenta el binario y prioriza entradas que descubren nueva cobertura.

### Referencia y ejemplo

Notas-10 (*overview syntax*) y notas-11; Manès et al., *The Art, Science, and Engineering of Fuzzing* (2019). *Ejemplo*: AFL encuentra *crashes* en parsers de PDF mutando un PDF válido y conservando las mutaciones que llegan a ramas nuevas.

### Pregunta 58

¿Por qué un *mutation fuzzer* suele ser más efectivo que un *random fuzzer*?

### Respuesta

Random fuzzing genera entradas completamente azarosas que rara vez pasan los filtros sintácticos de la unidad bajo prueba (un parser rechaza la entrada antes de tocar la lógica interesante). Mutation fuzzing parte de entradas válidas conocidas y aplica perturbaciones locales: las entradas resultantes preservan suficiente estructura para penetrar los filtros y a la vez exploran variaciones cercanas a casos legítimos. Por eso los fuzzers industriales modernos son todos mutation-based.

### Referencia y ejemplo

Notas-13 (*generación aleatoria*); Zalewski (AFL). *Ejemplo*: para una calculadora aritmética, random genera “xq#9a” (rechazada por el parser), mientras que mutation parte de “2+3” y produce “2++3”, explorando un caso real del parser.

### Pregunta 59

¿Qué es *differential testing* y cómo se combina con fuzzing?

### Respuesta

El *differential testing* ejecuta la misma entrada en dos implementaciones del mismo problema y reporta una falla si los resultados difieren. Una implementación se asume correcta (oráculo de referencia, p. ej. una calculadora estándar) y la otra es la unidad bajo prueba. Combinado con fuzzing, se generan miles de entradas y se compara automáticamente: cualquier discrepancia revela un bug en al menos una de las dos. Es la técnica detrás de CSmith (testing de compiladores C) y muchos validadores de protocolos.

**Referencia y ejemplo**

Notas-11; McKeeman, *Differential Testing for Software* (1998). *Ejemplo*: comparar el resultado de un evaluador propio con el comando UNIX `bc` para expresiones generadas por fuzzing.

**Pregunta 60**

¿Qué limitaciones tiene el *mutation score* como métrica?

**Respuesta**

Es un proxy, no una verdad. Limitaciones principales: **(1)** depende del catálogo de operadores; **(2)** los mutantes equivalentes inflan el numerador y exigen inspección manual; **(3)** no garantiza detectar defectos que el catálogo no modela (errores de spec, defectos de diseño); **(4)** es caro computacionalmente (la suite se corre una vez por mutante); **(5)** un score alto sobre código apenas cubierto no significa que la suite sea fuerte en el resto. Conviene combinarlo con cobertura estructural y revisar mutantes vivos uno a uno.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 9; Inozemtseva & Holmes, *Coverage Is Not Strongly Correlated with Test Suite Effectiveness* (2014). *Ejemplo*: dos suites con 95 % de mutation score pueden tener efectividad muy distinta si una cubre el 50 % del código y otra el 95 %.

## 7. Property-Based Testing (PBT)

### Temas abordados

- Propiedades como predicados universales sobre la entrada.
- Generadores aleatorios y composición.
- *Shrinking* de contraejemplos.
- Familias de propiedades: invariantes, round-trip, idempotencia, metamórficas.
- Relación de PBT con example-based testing y coverage-guided fuzzing.

### Preguntas y respuestas

**Pregunta 61**

¿Qué diferencia conceptual hay entre *example-based testing* y *property-based testing*?

**Respuesta**

En *example-based* el programador escribe casos concretos: tripla (entrada, acción, salida esperada). En *property-based* se enuncian *propiedades* (predicados universales) que la unidad debe cumplir para *toda* entrada válida; el framework genera automáticamente cientos de entradas aleatorias y reporta como falla cualquier contraejemplo, aplicando *shrinking* para minimizarlo. Son complementarios: example-based prueba casos puntuales bien elegidos; PBT cubre regiones del dominio automáticamente.

**Referencia y ejemplo**

Notas-12 (*pbt*); Claessen & Hughes, *QuickCheck: a lightweight tool for random testing of Haskell programs* (2000). *Ejemplo*: example-based: `assertEquals([1,2,3], sort([3,2,1]))`; PBT:



“ $\forall L : \text{sort}(L)$  es permutación ordenada de  $L$ ”.

### Pregunta 62

Enumere las familias típicas de propiedades en PBT.

### Respuesta

**Invariantes:** el resultado/estado cumple `repOK()` o una postcondición. **Round-trip:** `decode(encode(x))==x`. **Idempotencia:** `f(f(x))==f(x)`. **Conmutatividad/asociatividad:**  $x \text{ op } y = y \text{ op } x$ . **Oráculo de referencia:** la salida coincide con una implementación naive correcta. **Metamórficas:** relaciones entre entradas vinculadas (p. ej. “ordenar luego revertir es igual a ordenar descendente”). **Preservación:** la operación conserva alguna propiedad agregada (tamaño, suma, etc.).

### Referencia y ejemplo

Notas-12; Hillel Wayne, *Property-Based Testing*. *Ejemplo:* para `reverse`, idempotencia se enuncia como “`reverse(reverse(L))==L`”.

### Pregunta 63

¿Qué es un *Arbitrary* y cómo se componen generadores en jqwik?

### Respuesta

Un `Arbitrary<T>` es un generador aleatorio de valores de tipo `T`, con su política de *shrinking* asociada. Se componen con `Combinators.combine(...)`: dadas `Arbitrary` de los componentes, se devuelve una `Arbitrary` del compuesto, mapeando la tupla generada al constructor del tipo. `Arbitraries.integers().between(a,b)` acota un rango; `.list().ofMaxSize(n)` construye listas. Se marca con `@Provide` un método estático que retorne el `Arbitrary` y se referencia desde la propiedad por nombre.

### Referencia y ejemplo

Notas-13 (*generación aleatoria*); documentación de jqwik. *Ejemplo:* `Combinators.combine(xs, ys).as(Point::new)` produce un `Arbitrary<Point>` a partir de dos generadores de coordenadas.

### Pregunta 64

¿Qué es el *shrinking* y por qué es esencial en PBT?

### Respuesta

Cuando el generador encuentra un contraejemplo, suele ser grande y desordenado (lista de 437 enteros aleatorios). El *shrinking* reduce ese contraejemplo a una forma mínima que aún hace fallar la propiedad: para enteros, prueba valores cercanos a cero; para listas, intenta acortarlos; para objetos compuestos, simplifica recursivamente sus componentes. El resultado es un contraejemplo legible (“lista [0,0]”) que apunta directamente a la causa raíz. Sin *shrinking*, debugging un fallo de PBT sería prácticamente imposible.

### Referencia y ejemplo

Notas-12; Hughes, *Experiences with QuickCheck* (2016). *Ejemplo:* jqwik reduce un caso “lista de 50 elementos negativos” a “lista [-1]”.

**Pregunta 65**

¿Qué son las *propiedades metamórficas* y cuándo conviene usarlas?

**Respuesta**

Una *propiedad metamórfica* es una relación entre dos ejecuciones de la unidad bajo prueba con entradas relacionadas: dada una entrada  $x$  y una transformación  $T$ , se espera  $f(T(x)) = g(f(x))$  para alguna relación  $g$  conocida. Conviene cuando *no hay oráculo de salida exacto* (p. ej. algoritmos de ML, simulación, optimización): no se sabe el resultado correcto, pero se conocen propiedades de transformación.

**Referencia y ejemplo**

Chen et al., *Metamorphic Testing* (1998); notas-12. *Ejemplo*: para un buscador, “buscar ‘hola’ debe devolver al menos los resultados que ‘hola mundo’ ” (relación de inclusión) es metamórfica.

**Pregunta 66**

¿Cómo se usa `repOK()` dentro de una propiedad?

**Respuesta**

Se construye un generador de *secuencias de operaciones* sobre el objeto bajo prueba. La propiedad ejecuta cada secuencia y, después de cada operación, invoca `repOK()` sobre el objeto y exige que devuelva `true`. Si una secuencia produce un estado inválido, el framework la reporta y la *shrunk* a la subsecuencia mínima. Esta combinación cubre miles de escenarios de uso con un único enunciado.

**Referencia y ejemplo**

Notas-12; jqwik (*Action-based properties*). *Ejemplo*: para una cola circular, generar secuencias de `enqueue/dequeue` y exigir `repOK()` tras cada paso.

**Pregunta 67**

¿Cuándo PBT no es adecuado y qué alternativa conviene?

**Respuesta**

PBT exige que el programador sepa *enunciar propiedades correctas y observables*. No conviene cuando: **(1)** la unidad no tiene propiedades evidentes (lógica de negocio idiosincrática sin invariantes claras); **(2)** el oráculo es “el cliente lo aceptó” (validación); **(3)** el espacio de entradas es trivial y unos pocos ejemplos son suficientes; **(4)** la propiedad es probabilística (no `true/false`). En esos casos se prefieren example-based o testing manual.

**Referencia y ejemplo**

Notas-12. *Ejemplo*: validar el formato visual de una factura PDF es difícil de enunciar como propiedad; conviene un test de comparación con un PDF de referencia (example-based).

**Pregunta 68**

¿Qué relación hay entre PBT y coverage-guided fuzzing?

**Respuesta**

Ambos generan entradas automáticamente y usan oráculos genéricos (crashes, invariantes, propiedades). PBT enfatiza *propiedades semánticas* declaradas por el programador y opera a nivel de tipos; coverage-guided fuzzing enfatiza *maximizar cobertura* explorando el binario instrumentado y suele usar oráculos más débiles (crash/no crash). La frontera se difumina con herramientas modernas (jqwik puede guiarse por cobertura; libFuzzer admite propiedades como “`abort()` si falla un `assert`”).

**Referencia y ejemplo**

Notas-12, notas-13; Padhye et al., *JQF: Coverage-Guided Property-Based Testing in Java* (2019). *Ejemplo*: JQF combina jqwik con instrumentación al estilo AFL.

**Pregunta 69**

¿Cómo se elige una buena propiedad para una unidad dada?

**Respuesta**

Heurísticas: **(1)** buscar invariantes que la spec impone (resultado válido, tamaño, dominio); **(2)** buscar simetrías y relaciones algebraicas (asociatividad, inversa, idempotencia); **(3)** comparar con una implementación naive obviamente correcta; **(4)** aplicar transformaciones a la entrada y observar el efecto en la salida (metamórficas). Una buena propiedad es: simple de enunciar, falsable (el generador puede generar contraejemplos), y no trivial (no se cumple por construcción del código).

**Referencia y ejemplo**

Notas-12; Fink & Bishop, *Property-based testing for the masses*. *Ejemplo*: para `sort`, una propiedad fuerte es “el resultado está ordenado *y* es permutación de la entrada”; la primera sola se cumpliría con `return []`;

**Pregunta 70**

¿Cómo se controla el número de casos generados y la *seed* en PBT?

**Respuesta**

Frameworks como jqwik permiten configurar el número de tries (`@Property(tries=1000)`), la *seed* aleatoria (`@Property(seed="42")`) y la política de generación (*exhaustive* para dominios chicos vs. *random* para dominios grandes). Fijar la seed hace los runs reproducibles (esencial para debuggear contraejemplos); aumentar los tries aumenta la confianza, con costo lineal. La buena práctica es: muchos tries en CI nocturno, seed fija en CI rápida.

**Referencia y ejemplo**

Documentación de jqwik (*Property attributes*); notas-13. *Ejemplo*: `@Property(tries=10000, seed="1234")` ejecuta diez mil casos reproducibles.

## 8. Generación automática de tests y *mocking*

### Temas abordados

- Generación *feedback-directed random*: Randoop.

- Generación evolutiva guiada por cobertura: EvoSuite.
- Ejecución simbólica como técnica complementaria.
- Test doubles: *stub*, *mock*, *spy*, *fake*.
- Métricas combinadas: JaCoCo + PIT.

## Preguntas y respuestas

### Pregunta 71

¿Cómo funciona Randoop a alto nivel?

### Respuesta

Randoop es un generador *feedback-directed random*: parte de los constructores públicos y construye secuencias de llamadas a métodos. Mantiene un *pool* de objetos ya creados con resultados conocidos y, en cada iteración, extiende secuencias del pool agregando una nueva llamada con argumentos tomados del pool. Tras ejecutar cada secuencia, decide si la guarda como test *regression* (todo OK, captura la salida observada como oráculo) o como test *error-revealing* (violación de contrato: `equals/hashCode`, `toString`, excepciones inesperadas). Itera hasta agotar tiempo o tests.

### Referencia y ejemplo

Notas-15 (*evosuite/randoop*); Pacheco et al., *Feedback-Directed Random Test Generation* (ICSE 2007). *Ejemplo*: dada `Stack`, Randoop genera `Stack s=new Stack(); s.push(1); s.pop();` y guarda el resultado observado como `assert`.

### Pregunta 72

¿En qué se diferencia EvoSuite de Randoop?

### Respuesta

EvoSuite usa un *algoritmo genético* guiado por cobertura: parte de una población de suites, evalúa su *fitness* (cobertura de líneas, ramas, mutación) y aplica selección, cruzamiento y mutación para evolucionar suites mejores. Genera además *asserts* automáticos como oráculos. Randoop, en cambio, es aleatorio puro guiado por feedback. Randoop es más rápido y produce tests legibles; EvoSuite es más lento pero suele lograr mayor cobertura y mejores asserts.

### Referencia y ejemplo

Notas-15; Fraser & Arcuri, *EvoSuite: Automatic Test Suite Generation for Object-Oriented Software* (FSE 2011). *Ejemplo*: para una clase con un `if` difícil de alcanzar (`x==42`), EvoSuite evoluciona entradas hacia ese valor; Randoop puede no llegar nunca.

### Pregunta 73

¿Qué es la *ejecución simbólica* y cómo complementa la generación automática?

### Respuesta

La ejecución simbólica recorre los caminos del programa tratando las entradas como *variables simbólicas* y acumulando *path conditions* (fórmulas booleanas sobre las entradas). Al llegar a una rama, se invoca a un solver SMT (Z3) para encontrar valores concretos que la satisfagan. Genera entradas que ejercitan caminos específicos. Complementa a Randoop/EvoSuite cuando

los *guard conditions* son demasiado restrictivos para que un generador aleatorio los acierte por azar.

### Referencia y ejemplo

Notas-16 (*SymExec*); King, *Symbolic Execution and Program Testing* (1976); KLEE (Cadar et al., 2008). *Ejemplo*: para alcanzar `if(x==42)` con probabilidad razonable, ejecución simbólica resuelve  $x = 42$  en un paso; un fuzzer aleatorio tardaría  $\sim 2^{32}$  intentos.

### Pregunta 74

¿Qué es un *test double* y qué tipos existen?

### Respuesta

Un *test double* es un objeto que sustituye a una dependencia real durante un test. Tipos (Meszaros): **Dummy**: se pasa pero no se usa. **Fake**: implementación funcional simplificada (BD en memoria). **Stub**: devuelve respuestas predefinidas a llamadas, sin verificar interacción. **Mock**: además del stub, verifica que se invoque con argumentos y cantidades esperadas. **Spy**: objeto real al que se le observan las llamadas, preservando comportamiento salvo en los puntos interceptados.

### Referencia y ejemplo

Meszaros, *xUnit Test Patterns* (2007); notas-14 (*mocks*). *Ejemplo*: en lugar de conectar a Postgres, se inyecta un *fake* `HashMap`; para verificar que se llamó a `cobrar(monto)`, se usa un mock.

### Pregunta 75

¿Cuándo conviene usar un *mock* y cuándo es contraproducente?

### Respuesta

Conviene cuando la dependencia es lenta (red, BD), no determinista (reloj, RNG), tiene efectos colaterales no deseados (envío de mail), aún no está implementada o es difícil de poner en un estado concreto. Es contraproducente cuando: **(1)** se mockea la propia unidad bajo prueba (tautología); **(2)** el mock duplica la implementación real (el test pasa solo porque el mock dice lo mismo que el código); **(3)** se sobre-mockea, acoplando el test a detalles de implementación que cambiarán.

### Referencia y ejemplo

Notas-14; Fowler, *Mocks Aren't Stubs*. *Ejemplo*: mockear el reloj para testear “expira en 30 días” es correcto; mockear el resultado del método que se está testeando es absurdo.

### Pregunta 76

¿Qué información complementaria aportan JaCoCo y Pitest al evaluar una suite?

### Respuesta

**JaCoCo**: mide cobertura estructural (líneas, ramas, métodos). Da una idea del alcance. **Pitest**: mide *mutation score*: cuán fuertes son los oráculos. Una suite generada por Randoop puede tener alta cobertura pero bajo mutation score (oráculos débiles); una suite hecha a mano puede tener menos cobertura y mejor mutation score (asserts precisos). Usar ambas evita confiarse en una sola métrica: cobertura sin oráculos no detecta defectos.

### Referencia y ejemplo

Documentación de JaCoCo y Pitest; notas-15. *Ejemplo*: suite que logra 95 % line coverage con asserts solo de “no excepción” tendrá alta cobertura JaCoCo pero bajo mutation score: matará pocos mutantes porque sus asserts no distinguen valores.

### Pregunta 77

¿Por qué los *asserts* generados automáticamente pueden ser problemáticos?

### Respuesta

Tanto Randoop como EvoSuite generan asserts capturando la *salida observada* del código actual como “valor esperado”. Si el código tiene un bug, el assert codifica el bug y la suite resultante valida el comportamiento incorrecto en lugar del especificado. Esto se llama *regression oracle problem*. Estos asserts sirven bien para detectar *regresiones* futuras, no para verificar correctitud actual. El programador debe revisarlos y corregir manualmente los que codifican comportamientos no deseados.

### Referencia y ejemplo

Notas-15; Barr et al., *The Oracle Problem in Software Testing* (TSE 2015). *Ejemplo*: si cobrar actualmente devuelve un saldo incorrecto, el assert generado fijará ese valor como “esperado” y enmascarará el bug.

### Pregunta 78

¿Qué *test smells* comunes aparecen en suites generadas automáticamente y cómo se mitigan?

### Respuesta

Comunes: (1) nombres ininteligibles (`test_42`); (2) secuencias de llamadas largas y artificiales; (3) asserts redundantes o triviales (`assertNotNull` en cada objeto); (4) acoplamiento a detalles de implementación (`toString` exacto). Mitigaciones: usarlos como base de regresión, renombrar y simplificar manualmente los críticos, complementar con tests manuales bien nombrados, y considerar los autogenerados como “ruido organizado”.

### Referencia y ejemplo

Notas-15; van Deursen et al., *Refactoring Test Code*. *Ejemplo*: refactorizar `test_42` a `push_sobre_pila_llena_lanza_excepcion` mejora la mantenibilidad.

### Pregunta 79

¿Qué es *regression testing* y cómo se mantiene una suite de regresión a lo largo del tiempo?

### Respuesta

El *regression testing* es la re-ejecución sistemática de la suite ante cada cambio para detectar regresiones (defectos introducidos en código que antes funcionaba). Mantener la suite implica: eliminar tests obsoletos cuando cambia el contrato, actualizar oráculos cuando cambia la spec, distinguir fallas reales de *flakiness*, y priorizar la suite (*test prioritization*) ante runtime limitado. Las suites autogeneradas son útiles para la cobertura de regresión amplia.

### Referencia y ejemplo

Notas-15; Yoo & Harman, *Regression testing minimization, selection and prioritization* (2012). *Ejemplo*: tras refactorizar una API, los tests Randoop antiguos rompen masivamente; conviene

regenerar la base de regresión sobre la nueva versión estable.

#### Pregunta 80

¿Qué aporta combinar Randoop, EvoSuite, **repOK** y PBT en una misma campaña?

#### Respuesta

**Randoop**: suite de regresión amplia y rápida, detecta excepciones y violaciones de contratos básicos. **EvoSuite**: maximiza cobertura estructural y agrega asserts automáticos. **repOK**: oráculo interno que detecta corrupciones de estado invisibles desde la salida (cierra falsos negativos de Randoop/EvoSuite). **PBT**: enuncia propiedades de alto nivel y genera secuencias aleatorias estructuradas, cubriendo escenarios que las técnicas anteriores no exploran. La combinación reduce los *ángulos ciegos* de cada técnica individual.

#### Referencia y ejemplo

Notas-12, notas-14, notas-15. *Ejemplo*: para una clase **Server** con estado, Randoop encuentra una excepción no controlada, EvoSuite alcanza una rama difícil, PBT con **repOK** descubre que tras 50 operaciones aleatorias el estado interno viola un invariante.

## 9. Testing de sistemas con estado, no-determinismo e integración

### Temas abordados

- Testing de regresión y mantenimiento de suites.
- Diseño para la testabilidad: *dependency injection*.
- No-determinismo y reproducibilidad.
- Modelos de máquina de estados y testing basado en modelos.
- Métricas combinadas, requirements inviables y reporte de campañas.

### Preguntas y respuestas

#### Pregunta 81

¿Qué es la *testabilidad* de un diseño y qué prácticas la favorecen?

#### Respuesta

La *testabilidad* es la facilidad con la que se pueden escribir tests automatizados que ejerciten una unidad. Prácticas que la favorecen: **(1)** *Dependency Injection*: las dependencias se reciben como parámetros, no se instancian internamente; **(2)** separación de comandos (efectos) y consultas (sin efectos); **(3)** evitar singletons estáticos y estado global; **(4)** interfaces claras y mínimas (ISP/SRP); **(5)** pure functions cuando sea posible. Un diseño testeable produce, casi por casualidad, también código más mantenible y modular.

#### Referencia y ejemplo

Feathers, *Working Effectively with Legacy Code*; SOLID. *Ejemplo*: en lugar de `new Random()` adentro del método, recibir un `RandomGenerator` por constructor permite inyectar un stub determinista en tests.



**Pregunta 82**

¿Cómo se testean sistemas con *no-determinismo*?

**Respuesta**

Se controla la fuente del no-determinismo: **(1) reloj**: inyectar un `Clock` y avanzarlo manualmente; **(2) RNG**: inyectar un generador con semilla fija o stub; **(3) concurrencia**: usar primitivas de sincronización en tests o testing estocástico con muchas repeticiones; **(4) red/BD**: mockear o usar fakes; **(5) ordenamiento de colecciones**: no asumir orden, usar asserts sobre conjuntos. La regla general es: *aislar el azar* de modo que el resultado del test dependa solo de entradas controladas.

**Referencia y ejemplo**

Notas-03, notas-14; Beck, *Test-Driven Development by Example*. *Ejemplo*: para testear “el sorteo elige a Juan”, se inyecta un RNG que en su próximo `nextInt` devuelve exactamente el índice de Juan.

**Pregunta 83**

¿Qué es *Dependency Injection* y por qué facilita el testing?

**Respuesta**

Es el patrón de diseño según el cual una unidad recibe sus dependencias desde afuera (parámetros del constructor, setters o framework) en lugar de instanciarlas internamente. Facilita el testing porque permite reemplazar dependencias reales por *test doubles* sin modificar la unidad bajo prueba: el mismo código corre con la dependencia real en producción y con un mock/stub en test. Invierte el control y desacopla.

**Referencia y ejemplo**

Fowler, *Inversion of Control Containers and the Dependency Injection pattern*; notas-14. *Ejemplo*: `class OrderService(PaymentGateway gw)` permite testear con un mock de `PaymentGateway` sin tocar el código de `OrderService`.

**Pregunta 84**

¿Cuándo y por qué declarar un *requirement* de test como *inviabile*?

**Respuesta**

Cuando satisfacerlo no es posible o es desproporcionadamente caro: **(1)** requiere acceso a estado interno que no se puede observar sin instrumentación; **(2)** depende de comportamiento no determinista no controlable; **(3)** es semánticamente infactible (condiciones contradictorias); **(4)** corresponde a una garantía estadística (uniformidad de un RNG), no a un test unitario. Documentar requirements inviables explícitamente es buena práctica: previene falsas sensaciones de completitud y comunica los límites de la suite.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 2 (*Infeasible requirements*); notas-02. *Ejemplo*: “el constructor coloca la ficha en la posición  $(i, j)$  exacta” es inviable sin mockear el RNG; se declara inviable y se documenta.

**Pregunta 85**

¿Cómo se modela un sistema con estado para aplicar testing basado en modelos?

**Respuesta**

Se identifica el *conjunto de estados* relevantes (típicamente abstracciones del estado interno) y las *transiciones* disparadas por operaciones de la unidad. El resultado es una *máquina de estados finita* (FSM). Sobre la FSM se aplican criterios análogos a los del CFG: *state coverage* (visitar cada estado), *transition coverage* (recorrer cada transición), *transition-pair coverage*, etc. Las secuencias de operaciones para satisfacer cada criterio son los tests.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 7 (*Graphs from other sources*); Utting & Legeard, *Practical Model-Based Testing*. *Ejemplo*: una sesión de login se modela con estados {anónimo, autenticado, bloqueado} y transiciones {login\_ok, login\_fail, logout}.

**Pregunta 86**

¿Qué métricas se combinan para evaluar la *calidad* de una suite de pruebas?

**Respuesta**

No hay una métrica única. Se combinan: **cobertura estructural** (líneas, ramas, métodos: JaCoCo), **mutation score** (Pitest: fortaleza de oráculos), **cobertura funcional** (qué fracción de los requirements de spec se cubren), **cantidad de defectos detectados** en datos históricos, **tiempo de ejecución** (suite rápida  $\Rightarrow$  se corre seguido), y **flakiness rate** (fracción de tests no reproducibles). Una suite “buena” es alta en las primeras y baja en las últimas dos.

**Referencia y ejemplo**

Ammann & Offutt, Cap. 2; Inozemtseva & Holmes (2014). *Ejemplo*: 95 % cobertura + 90 % mutation score + 0 flaky + 5s de runtime es mejor suite que 99 % cobertura + 50 % mutation score + flakiness alta.

**Pregunta 87**

¿Qué es el *testing exploratorio* y cuándo se combina con tests automatizados?

**Respuesta**

El *testing exploratorio* es una práctica donde el tester explora libremente el sistema, sin guion preestablecido, buscando defectos guiado por intuición y experiencia. Combina aprendizaje y ejecución: cada hallazgo guía la siguiente exploración. Se combina con tests automatizados así: los automatizados cubren regresión y casos conocidos; el exploratorio descubre defectos imprevistos, especialmente en UI y experiencia de usuario, y los hallazgos se convierten en nuevos tests automatizados.

**Referencia y ejemplo**

Kaner, Bach & Pettichord, *Lessons Learned in Software Testing*; notas-00. *Ejemplo*: tras encontrar exploratoriamente un crash al subir un archivo de 0 bytes, se agrega un test automatizado parametrizado con ese caso.

**Pregunta 88**

¿Qué es el *test rot* y cómo se previene?

**Respuesta**

El *test rot* es la degradación progresiva de una suite: tests que se vuelven obsoletos cuando cambia la spec, tests *flaky* ignorados, tests con asserts laxos, tests acoplados a detalles de implementación que rompen ante refactors menores. Se previene con: **(1)** revisión periódica de la suite; **(2)** eliminación inmediata de tests obsoletos; **(3)** cero tolerancia a flakiness (un flaky se arregla o se elimina); **(4)** testear comportamiento, no implementación; **(5)** buenas convenciones de nombrado para facilitar el mantenimiento.

**Referencia y ejemplo**

Feathers, *Working Effectively with Legacy Code*. *Ejemplo*: si un assert verifica un mensaje de log exacto, cambiar el texto del log rompe el test sin que haya bug real: ese assert está acoplado a implementación y debería verificar comportamiento (¿el log se emitió?), no contenido literal.

**Pregunta 89**

¿Cómo se relacionan las técnicas de testing entre sí y cuándo conviene cada una?

**Respuesta**

No son exclusivas, son complementarias y atacan defectos de naturaleza distinta. **ISP**: cuando la unidad tiene parámetros con dominios particionables. **Grafos**: cuando hay flujo de control no trivial. **Lógica de predicados**: cuando hay decisiones compuestas (típico de software crítico). **Mutación**: para evaluar la fortaleza de los oráculos. **PBT**: cuando hay propiedades algebraicas o invariantes. **Generación automática**: para regresión amplia. **Mocks**: para aislar dependencias. Una campaña madura las orchestra según el riesgo y el costo.

**Referencia y ejemplo**

Ammann & Offutt, libro completo; notas del curso (00–16). *Ejemplo*: para un módulo de pago, ISP en validación de monto, CACC en reglas anti-fraude, mocks en gateway externo, PBT en propiedades de conservación de saldo y mutación para evaluar la fortaleza global.

**Pregunta 90**

¿Qué debe incluir el reporte final de una *campaña de testing*?

**Respuesta**

Buenas prácticas: **(1)** *alcance*: qué se testeó y qué no; **(2)** *criterios aplicados*: ISP, NC, CACC, mutación, PBT; **(3)** *requirements derivados y satisfechos*; **(4)** *requirements inviables* declarados con justificación; **(5)** *métricas*: cobertura JaCoCo, mutation score, número de tests, fallas detectadas; **(6)** *defectos encontrados y su severidad*; **(7)** *decisiones de diseño* (inyecciones, refactors hechos para testear); **(8)** *trabajo futuro*: ángulos no cubiertos. El reporte hace *auditable* el trabajo y evita la falsa sensación de completitud.

**Referencia y ejemplo**

ISO/IEC/IEEE 29119-3 (*Test Documentation*); notas-02. *Ejemplo*: un reporte típico incluye una tabla con cobertura por clase, un listado de los bugs encontrados con su trazabilidad al test que los reveló y la lista de requirements inviables con motivos.

## 10. Resumen de cobertura

| Unidad       | Tema principal                                                                                                | Preguntas |
|--------------|---------------------------------------------------------------------------------------------------------------|-----------|
| TP1          | Vocabulario del testing ( <i>fault/error/failure</i> ), V&V, RIPR, AAA, contrato <code>equals/hashCode</code> | 10        |
| TP2          | Automatización con JUnit, data-driven testing, fixtures, <code>rep0K</code> y reproducibilidad                | 10        |
| TP3          | Input Space Partitioning: MDE, particionado por equivalencia, EC / PWC / BCC / ACC, restricciones             | 10        |
| TP4          | Testing basado en grafos: CFG, NC / EC / EPC / PPC, tour/sidetrip/detour, complejidad ciclomática             | 10        |
| TP5          | Cobertura lógica: PC, CC, GACC / CACC / RACC, CoC, relación con MCDC y cortocircuito                          | 10        |
| TP6          | Testing de mutación, hipótesis del programador competente, <i>coupling</i> , Pitest y <i>fuzzing</i>          | 10        |
| TP7          | Property-Based Testing: propiedades, generadores, <i>shrinking</i> , propiedades metamórficas                 | 10        |
| TP8          | Generación automática (Randoop, EvoSuite), ejecución simbólica, <i>test doubles</i> , JaCoCo+PIT              | 10        |
| TP9          | Testabilidad, no-determinismo, modelos de estados, requirements inviables y reporte de campaña                | 10        |
| <b>Total</b> |                                                                                                               | <b>90</b> |

**Verificación:**  $10 \times 9 = 90$  preguntas teóricas, distribuidas uniformemente entre las nueve unidades de la materia.

**Nota sobre referencias:** las citas al libro hacen referencia a P. Ammann & J. Offutt, *Introduction to Software Testing* (Cambridge University Press, 2ª edición); las menciones a “notas-XX” son las notas de clase de la cátedra (V. Bengolea). Cuando la fuente exacta no es identificable en las notas, se cita la referencia académica más cercana de la disciplina (Bloch, Meszaros, Fowler, Feathers, Claessen & Hughes, McCabe, DeMillo et al.).